observer.bpf.c and controller.py, Explained

eBPF bufferbloat detection + adaptive AQM prototype | CSC 303, IIT Roorkee

1. How the two programs fit together

The prototype is two programs. `observer.bpf.c` is C code that runs **inside the Linux kernel** as eBPF. It only measures. `controller.py` is Python that runs in **user space**. It loads the eBPF program, reads what it measured, decides, and changes the qdisc with `tc`. The two share data through **BPF maps**: tables in kernel memory that both sides can read.

```
KERNEL (observer.bpf.c)                              USER SPACE (controller.py)
-----------------------------------------            ----------------------------------------
packet enters qdisc  --> net_dev_queue probe         Observer   loads the C file with BCC,
                         stamp time in enq_ts                   reads 'slots' and 'flows' maps
                         add bytes to 'flows'        Qdisc      reads tc stats, runs tc replace
packet leaves qdisc  --> net_dev_start_xmit probe    PingProbe  ping every 0.2 s, RTT inflation
                         sojourn = now - stamp       Controller state machine: decide + act
                         fold into 100 ms 'slots'    main()     0.5 s loop: measure, step, log
                  maps: enq_ts | flows | slots  <====  read every tick
```

Sojourn time means how long one packet waited inside the queue: the time it left minus the time it entered. It is the direct measure of bufferbloat. RTT from ping only shows the problem indirectly.

Control loop: measure (eBPF + tc + ping) -> detect (standing queue) -> decide (state machine) -> adapt (tc qdisc replace) -> verify (keep measuring, escalate or roll back).

2. observer.bpf.c - the eBPF program (kernel side)

134 lines of restricted C. BCC compiles it with clang to eBPF bytecode when the controller starts. The kernel **verifier** checks it is safe (no unbounded loops, no bad memory access) and then attaches it to two kernel tracepoints. There is no `main()`. The two `TRACEPOINT_PROBE` functions run automatically for every packet.

2.1 Header comment and compile-time constants (observer.bpf.c lines 1-11)

```
 1  // eBPF observer for one egress interface (loaded by controller.py through BCC).
 2  //
 3  //   net:net_dev_queue      packet handed to the qdisc   -> stamp it, count its flow
 4  //   net:net_dev_start_xmit packet leaves the qdisc      -> sojourn = now - stamp
 5  //
 6  // Sojourn time is therefore measured per packet, for whichever qdisc is
 7  // installed (pfifo included), and aggregated in the kernel into 100 ms slots.
 8  // User space only ever reads the slots and the per-flow table.
 9  //
10  // NAME_LO/HI and MASK_LO/HI (the interface name as two u64s), WINDOW_NS and
11  // NSLOTS are passed in as -D flags.
```

- Explains the idea: stamp a packet when it is handed to the qdisc (`net_dev_queue`), and compute sojourn when it leaves (`net_dev_start_xmit`).
- Because these tracepoints are *outside* the qdisc, the measurement works for **any** qdisc. This includes plain pfifo, which keeps no delay statistics of its own.
- Aggregation happens in the kernel. User space reads small summaries, not every packet, so the overhead stays low.
- `NAME_LO/HI`, `MASK_LO/HI`, `WINDOW_NS` and `NSLOTS` are not defined in this file. `controller.py` passes them as `-D` compiler flags (section 3.4), so the same C file works for any interface.

2.2 Kernel headers (observer.bpf.c lines 13-17)

```
13  #include <linux/skbuff.h>
14  #include <linux/netdevice.h>
15  #include <linux/ip.h>
16  #include <linux/in.h>
17  #include <linux/if_ether.h>
```

- Give the C code the kernel's definitions of `struct sk_buff` (a packet), `struct net_device` (an interface), `struct iphdr` (IPv4 header), protocol numbers (`IPPROTO_TCP`) and `ETH_P_IP`.

2.3 Data structures (observer.bpf.c lines 19-37)

```
19  struct flow_key {
20      u32 saddr;
21      u32 daddr;
22      u16 sport;
23      u16 dport;
24      u32 proto;
25  };
26
27  struct flow_val {
28      u64 bytes;
29  };
30
31  struct slot {
32      u64 epoch;      // which WINDOW_NS interval this slot currently holds
33      u64 count;
34      u64 sum_ns;
35      u64 min_ns;
36      u64 max_ns;
37  };
```

- `flow_key`: identifies a flow by its **five-tuple**: source IP, destination IP, source port, destination port, protocol.
- `flow_val`: what is stored per flow, the total bytes seen so far.
- `slot`: statistics for one 100 ms window: `count` packets, `sum_ns` (for the average), `min_ns` and `max_ns` sojourn. `epoch` records *which* 100 ms window the slot currently holds (the time divided by 100 ms). The reader uses it to tell fresh data from stale data.

2.4 BPF maps (shared with Python) (observer.bpf.c lines 39-41)

```
39  BPF_TABLE("lru_hash", u64, u64, enq_ts, 16384);                      // skb -> enqueue time
40  BPF_TABLE("lru_hash", struct flow_key, struct flow_val, flows, 4096); // five-tuple -> counters
41  BPF_ARRAY(slots, struct slot, NSLOTS);
```

- `enq_ts`: packet address -> enqueue timestamp. The key is the `sk_buff` pointer, which is unique while the packet is alive. It is an **LRU** hash, so if a packet is dropped inside the qdisc and never leaves, its old entry is evicted automatically instead of filling the table.
- `flows`: five-tuple -> byte counter, LRU with 4096 entries. Python reads this to show the "top flow" column.
- `slots`: an array of 64 `slot`s used as a **ring buffer**. 64 x 100 ms = 6.4 s of history, far more than the 0.5 s between controller reads.

2.5 on_target(): is this packet on our interface? (observer.bpf.c lines 43-53)

```
43  static __always_inline int on_target(struct sk_buff *skb)
44  {
45      struct net_device *dev = NULL;
46      u64 name[2] = {};
47
48      bpf_probe_read_kernel(&dev, sizeof(dev), &skb->dev);
49      if (!dev)
50          return 0;
51      bpf_probe_read_kernel(&name, sizeof(name), &dev->name);
52      return (name[0] & MASK_LO) == NAME_LO && (name[1] & MASK_HI) == NAME_HI;
53  }
```

- Tracepoints fire for **every** packet on **every** interface in **every** namespace. This function keeps only packets going out of the bottleneck interface (r-eth1).
- `bpf_probe_read_kernel` is the safe way to read kernel memory from eBPF. The verifier will not allow plain pointer dereferences here.

- It reads the packet's device pointer, then the 16-byte device name. It compares the name as two 64-bit numbers against `NAME_LO/HI` with masks. That is much cheaper than a string compare.
- The mask covers the name **plus its terminating zero byte**, so `r-eth1` does not accidentally match `r-eth10`.

2.6 count_flow(): per-flow byte counting (observer.bpf.c lines 55-85)

```
55  static __always_inline void count_flow(struct sk_buff *skb, u32 len)
56  {
57      unsigned char *head = NULL;
58      u16 nh_off = 0, proto = 0;
59      struct iphdr ip = {};
60      struct flow_key key = {};
61      struct flow_val zero = {}, *val;
62
63      bpf_probe_read_kernel(&proto, sizeof(proto), &skb->protocol);
64      if (proto != htons(ETH_P_IP))
65          return;
66      bpf_probe_read_kernel(&head, sizeof(head), &skb->head);
67      bpf_probe_read_kernel(&nh_off, sizeof(nh_off), &skb->network_header);
68      if (bpf_probe_read_kernel(&ip, sizeof(ip), head + nh_off) < 0)
69          return;
70
71      key.saddr = ip.saddr;
72      key.daddr = ip.daddr;
73      key.proto = ip.protocol;
74      if (ip.protocol == IPPROTO_TCP || ip.protocol == IPPROTO_UDP) {
75          u16 ports[2] = {};
76          bpf_probe_read_kernel(&ports, sizeof(ports), head + nh_off + ip.ihl * 4);
77          key.sport = ntohs(ports[0]);
78          key.dport = ntohs(ports[1]);
79      }
80
81      val = flows.lookup_or_try_init(&key, &zero);
82      if (!val)
83          return;
84      __sync_fetch_and_add(&val->bytes, len);
85  }
```

- Lines 63-65: only IPv4 packets (`ETH_P_IP`) are counted. Others return early.
- Lines 66-69: find the IP header. `skb->head` is the start of the packet buffer and `network_header` is the offset of the IP header within it. Copy the header into `ip`.
- Lines 71-79: fill the five-tuple. For TCP/UDP, the ports are the first 4 bytes after the IP header, which sits at offset `ihl * 4` because `ihl` is the header length in 32-bit words. `ntohs` converts from network to host byte order. ICMP (ping) has no ports, so they stay 0.
- Lines 81-84: find the flow's entry, or create it at zero, then add the packet length. `__sync_fetch_and_add` is an **atomic** add, because packets can be enqueued on several CPUs at the same moment.

2.7 Probe 1: packet ENTERS the qdisc (net_dev_queue) (observer.bpf.c lines 87-98)

```
87  TRACEPOINT_PROBE(net, net_dev_queue)
88  {
89      struct sk_buff *skb = (struct sk_buff *)args->skbaddr;
90      u64 key = (u64)skb, now;
91
92      if (!on_target(skb))
93          return 0;
94      now = bpf_ktime_get_ns();
95      enq_ts.update(&key, &now);
96      count_flow(skb, args->len);
97      return 0;
98  }
```

- Runs when the kernel hands a packet to the qdisc. `args` is the tracepoint's data. `args->skbaddr` is the packet's address.
- Ignore packets that are not on r-eth1. Otherwise take the time with `bpf_ktime_get_ns()` (a monotonic clock in nanoseconds) and store it in `enq_ts` under the packet's address.

- Also count the packet's bytes for its flow (`args->len` is the packet length).

2.8 Probe 2: packet LEAVES the qdisc (net_dev_start_xmit) (observer.bpf.c lines 100-134)

```
100  TRACEPOINT_PROBE(net, net_dev_start_xmit)
101  {
102      u64 key = (u64)args->skbaddr, now, sojourn, epoch;
103      u64 *tsp = enq_ts.lookup(&key);
104      struct slot *s;
105      int idx;
106
107      if (!tsp)
108          return 0;
109      now = bpf_ktime_get_ns();
110      sojourn = now - *tsp;
111      enq_ts.delete(&key);
112
113      epoch = now / WINDOW_NS;
114      idx = epoch % NSLOTS;
115      s = slots.lookup(&idx);
116      if (!s)
117          return 0;
118      // Dequeue for one device queue is serialised by the kernel, so the slot
119      // can be updated without atomics.
120      if (s->epoch != epoch) {
121          s->epoch = epoch;
122          s->count = 0;
123          s->sum_ns = 0;
124          s->min_ns = sojourn;
125          s->max_ns = sojourn;
126      }
127      s->count += 1;
128      s->sum_ns += sojourn;
129      if (sojourn < s->min_ns)
130          s->min_ns = sojourn;
131      if (sojourn > s->max_ns)
132          s->max_ns = sojourn;
133      return 0;
134  }
```

- Lines 103-108: look up the enqueue stamp for this packet. If there is none (a different interface, or a packet that was already queued before the program loaded), do nothing. This lookup is how the second probe filters packets without calling `on_target` again.
- Lines 109-111: **sojourn = now - enqueue time**, the time the packet spent in the queue. Then delete the stamp because it is no longer needed.
- Lines 113-117: work out which 100 ms window this is (`epoch = now / WINDOW_NS`) and which ring slot holds it (`idx = epoch % NSLOTS`).
- Lines 120-126: if the slot still holds an *older* window from 6.4 s ago, reset it for the new window. The first packet sets min and max to its own sojourn.
- Lines 127-132: update count, sum, min and max.
- Comment at 118-119: the kernel sends packets out of one device queue one at a time, so updates here cannot race and no atomics are needed. This differs from the enqueue side in 2.6.

3. controller.py - the controller (user space)

372 lines of Python, run as root. It has four classes and a `main()`: `Qdisc` (talks to tc), `Observer` (loads and reads the eBPF program), `PingProbe` (measures RTT), `Controller` (the decision logic), and `main()` (wires them together and runs the 0.5 s loop).

3.1 Docstring, imports, constants (controller.py lines 1-30)

```
#!/usr/bin/env python3
"""Bufferbloat detector + adaptive AQM controller (prototype).

    observe  eBPF sojourn time and per-flow counters, qdisc statistics, a ping probe
    detect   saturated link + standing queue + RTT inflation, held for a few seconds
    decide   NORMAL -> DEGRADED -> MITIGATING -> RECOVERED, with hysteresis and dwell
    act      replace the leaf qdisc on the bottleneck: pfifo -> fq_codel -> cake
    verify   keep measuring; escalate or roll back if the change did not help

Run as root. Every threshold below is an illustrative default, not a tuned value.
"""
import argparse
import collections
import csv
import json
import os
import re
import signal
import socket
import statistics
import struct
import subprocess
import sys
import threading
import time

LADDER = ["pfifo", "fq_codel", "cake"]
WINDOW_NS = 100_000_000     # sojourn aggregation slot, matches CoDel's interval
NSLOTS = 64
HERE = os.path.dirname(os.path.abspath(__file__))
```

- The docstring is the design in five lines: observe, detect, decide, act, verify.
- `LADDER`: the qdiscs in order of strength. The controller only moves one rung at a time.
- `WINDOW_NS` = 100 ms: the eBPF aggregation window. It matches CoDel's interval, the standard time scale for telling a burst from a standing queue. `NSLOTS` = 64 ring slots.
- These two constants are passed to the C code, so both sides always agree.

3.2 sh(): run a shell command (controller.py lines 33-34)

```
def sh(cmd):
    return subprocess.run(cmd, check=True, capture_output=True, text=True).stdout
```

- Runs a command and returns its output. `check=True` raises an error if the command fails, so a failed `tc` call is never silently ignored.

3.3 class Qdisc: read stats and swap the qdisc (controller.py lines 37-62)

```
37  class Qdisc:
38      """Reads statistics from, and replaces, the leaf qdisc under the shaper."""
39
40      def __init__(self, ns, dev, parent, pfifo_limit):
41          self.tc = ["ip", "netns", "exec", ns, "tc"] if ns else ["tc"]
42          self.dev, self.parent, self.pfifo_limit = dev, parent, pfifo_limit
43          self.handle = 100
44
45      def stats(self):
46          root = leaf = None
47          for q in json.loads(sh(self.tc + ["-s", "-j", "qdisc", "show", "dev", self.dev])):
48              if q.get("root"):
49                  root = q
50              elif q.get("parent") == self.parent:
51                  leaf = q
52          if root is None or leaf is None:
53              raise RuntimeError(f"no shaper + leaf qdisc on {self.dev}; run testbed.sh up first")
54          return root, leaf
55
56      def apply(self, kind):
57          # A fresh handle each time: tc refuses to change a qdisc's kind in place.
58          # Note that replacing a qdisc discards whatever was queued in the old one.
59          self.handle += 1
60          args = ["limit", str(self.pfifo_limit)] if kind == "pfifo" else []
61          sh(self.tc + ["qdisc", "replace", "dev", self.dev, "parent", self.parent,
62                        "handle", f"{self.handle}:", kind] + args)
```

- Line 41: every `tc` command is run inside the router namespace (`ip netns exec bb-r tc ...`).
- `stats()`: runs `tc -s -j qdisc show` (JSON output). It finds the **root** qdisc (the htb 10 Mbit shaper, whose byte counter gives the link rate) and the **leaf** qdisc under class `1:1` (the one we manage, whose backlog and drops we read). If either is missing, the testbed is not up and it raises an error.
- `apply(kind)`: **this is the "adapt" step**. It runs `tc qdisc replace dev r-eth1 parent 1:1 handle N: fq_codel`. It uses a new handle each time because tc will not change a qdisc's type in place. pfifo is given the 1000-packet limit so the base case is a deep, bloated buffer.
- Comment line 58: replacing the qdisc throws away packets queued in the old one. That is acceptable because those packets were stuck in a bloated queue anyway.

3.4 class Observer: load the eBPF program (controller.py lines 65-85)

```
65  class Observer:
66      """Loads observer.bpf.c and reads its maps."""
67
68      def __init__(self, dev):
69          from bcc import BPF
70          name = dev.encode()
71          if len(name) > 15:
72              raise ValueError("interface name too long")
73          padded = name.ljust(16, b"\0")
74          mask = (b"\xff" * (len(name) + 1)).ljust(16, b"\0")
75          u64 = lambda b: f"0x{int.from_bytes(b, 'little'):x}ULL"
76          self.bpf = BPF(src_file=os.path.join(HERE, "observer.bpf.c"), cflags=[
77              "-Wno-macro-redefined",
78              f"-DNAME_LO={u64(padded[:8])}", f"-DNAME_HI={u64(padded[8:])}",
79              f"-DMASK_LO={u64(mask[:8])}", f"-DMASK_HI={u64(mask[8:])}",
80              f"-DWINDOW_NS={WINDOW_NS}ULL", f"-DNSLOTS={NSLOTS}",
81          ])
82          self.slots = self.bpf["slots"]
83          self.flows = self.bpf["flows"]
84          self.next_epoch = time.monotonic_ns() // WINDOW_NS
85          self.prev_bytes = {}
```

- Line 69: imports `bcc`, the library that compiles and loads eBPF from Python.
- Lines 70-75: turn the interface name (e.g. `r-eth1`) into the `NAME_LO/HI` and `MASK_LO/HI` numbers that `on_target()` compares against. Linux limits interface names to 15 characters, plus a zero byte, which is 16 bytes or two 64-bit values.

- Lines 76-81: **this line loads observer.bpf.c into the kernel**. `BPF(src_file=...)` compiles it with the `-D` constants, the verifier checks it, and the two `TRACEPOINT_PROBE`s are attached automatically.
- Lines 82-83: handles to the `slots` and `flows` maps, which Python can read like dicts.
- Line 84: `next_epoch` remembers which 100 ms window to read next. Python's `time.monotonic_ns()` uses the same clock as `bpf_ktime_get_ns()`, so the window numbers line up exactly.

3.5 Observer.sojourn(): read the delay signal (controller.py lines 87-109)

```
 87      def sojourn(self):
 88          """(min, avg, max) in ms over the 100 ms windows completed since the last call.
 89
 90          min is the smallest per-window minimum. A window with no departures counts
 91          as zero: nothing was waiting. So min > X means that in every window, every
 92          packet waited longer than X, which is CoDel's definition of a standing queue.
 93          """
 94          now_epoch = time.monotonic_ns() // WINDOW_NS
 95          epochs = range(max(self.next_epoch, now_epoch - NSLOTS + 2), now_epoch)
 96          self.next_epoch = now_epoch
 97          mins, count, total, peak = [], 0, 0, 0
 98          for epoch in epochs:
 99              s = self.slots[epoch % NSLOTS]
100              if s.epoch != epoch or s.count == 0:
101                  mins.append(0)
102                  continue
103              mins.append(s.min_ns)
104              count += s.count
105              total += s.sum_ns
106              peak = max(peak, s.max_ns)
107          if not mins:
108              return None
109          return min(mins) / 1e6, (total / count / 1e6 if count else 0.0), peak / 1e6
```

- Line 95: the windows to read are those completed since the last call, up to but **not** including the current, still-filling window. It never reaches back further than the ring holds.
- Line 100: if a slot's `epoch` does not match, or it is empty, no packet left in that 100 ms. That counts as 0 delay, because nothing was waiting.
- Returns (min, avg, max) in ms. The controller uses the **min**, and that is the key idea. If even the *smallest* per-window minimum is above 50 ms, then **every packet in every window** waited more than 50 ms. That is a **standing queue** (CoDel's definition), not a short burst that drains by itself.

3.6 Observer.top_flows(): who is using the link (controller.py lines 111-124)

```
111      def top_flows(self, dt, n=3):
112          rates, seen = [], {}
113          for k, v in self.flows.items():
114              key = (k.saddr, k.daddr, k.sport, k.dport, k.proto)
115              seen[key] = v.bytes
116              delta = v.bytes - self.prev_bytes.get(key, 0)
117              if delta > 0:
118                  rates.append((delta * 8 / dt / 1e6, key))
119          self.prev_bytes = seen
120          ip = lambda a: socket.inet_ntoa(struct.pack("I", a))
121          proto = {1: "icmp", 6: "tcp", 17: "udp"}
122          return len(seen), [
123              f"{proto.get(k[4], k[4])} {ip(k[0])}:{k[2]}>{ip(k[1])}:{k[3]} {mbit:.1f}Mbit"
124              for mbit, k in sorted(rates, reverse=True)[:n]]
```

- Reads every entry in the `flows` map. It subtracts the byte count from the previous call and divides by the elapsed time to get each flow's rate in Mbit/s.
- Formats the top 3 as e.g. `tcp 10.0.1.2:45678>10.0.2.2:5201 9.5Mbit` for the live table. This is for display and diagnosis. It does not affect decisions.

3.7 class PingProbe: RTT of an interactive "victim" flow (controller.py lines 127-164)

```
127  class PingProbe(threading.Thread):
128      """Interactive 'victim' flow: 5 pings/s across the bottleneck."""
129
130      def __init__(self, ns, target, t0, log_path, baseline_window):
131          super().__init__(daemon=True)
132          cmd = ["ping", "-n", "-i", "0.2", target]
133          self.proc = subprocess.Popen((["ip", "netns", "exec", ns] if ns else []) + cmd,
134                                       stdout=subprocess.PIPE, stderr=subprocess.DEVNULL, text=True)
135          self.t0, self.lock, self.fresh = t0, threading.Lock(), []
136          self.history = collections.deque()
137          self.baseline_window = baseline_window
138          self.log = open(log_path, "w", newline="")
139          self.writer = csv.writer(self.log)
140          self.writer.writerow(["t", "rtt_ms"])
141
142      def run(self):
143          for line in self.proc.stdout:
144              m = re.search(r"time=([\d.]+) ms", line)
145              if not m:
146                  continue
147              t, rtt = time.monotonic() - self.t0, float(m.group(1))
148              self.writer.writerow([f"{t:.3f}", rtt])
149              with self.lock:
150                  self.fresh.append(rtt)
151                  self.history.append((t, rtt))
152
153      def sample(self, now):
154          """(median RTT since last call, baseline = windowed minimum RTT), or (None, baseline)."""
155          with self.lock:
156              fresh, self.fresh = self.fresh, []
157              while self.history and self.history[0][0] < now - self.baseline_window:
158                  self.history.popleft()
159              base = min((r for _, r in self.history), default=None)
160          return (statistics.median(fresh) if fresh else None), base
161
162      def stop(self):
163          self.proc.terminate()
164          self.log.flush()
```

- A background thread that runs `ping -i 0.2 10.0.2.2` from namespace bb-h1 (5 pings per second across the bottleneck). It stands in for latency-sensitive traffic such as a game or video call.
- `run()`: reads ping's output line by line, extracts `time=XX ms` with a regex, writes it to `*.ping.csv`, and stores it under a lock because two threads share the data.
- `sample()`: returns the **median** RTT since the last call, and a **baseline**: the minimum RTT over the last 120 s (the path's "empty-queue" RTT, about 20 ms here).
- **Inflation** = current RTT - baseline is the extra delay caused by queueing. It is the end-to-end check that the eBPF signal reflects real harm to users.

3.8 class Controller: state and helpers (controller.py lines 167-195)

```
167  class Controller:
168      def __init__(self, args, qdisc, emit):
169          self.a, self.qdisc, self.emit = args, qdisc, emit
170          self.state = "NORMAL"
171          self.rung = LADDER.index(args.base)
172          self.policy = args.base
173          self.recent = collections.deque(maxlen=64)   # (t, delay_ms)
174          self.bad_since = self.good_since = self.idle_since = None
175          self.changed_at = self.locked_until = 0.0
176          self.before = None
177          self.prev_rung = self.rung
178          self.told_top = False
179
180      def _median_delay(self, since):
181          vals = [d for t, d in self.recent if t >= since]
182          return statistics.median(vals) if vals else 0.0
183
184      def _switch(self, rung, now, why):
185          old = self.policy
186          self.before = self._median_delay(now - 3)
187          self.prev_rung, self.rung, self.policy = self.rung, rung, LADDER[rung]
188          self.qdisc.apply(self.policy)
189          self.changed_at, self.told_top = now, False
190          self.bad_since = self.good_since = self.idle_since = None
191          self.emit(f"qdisc {old} -> {self.policy}  ({why})")
192
193      def _goto(self, state, why):
194          self.emit(f"state {self.state} -> {state}  ({why})")
195          self.state = state
```

- State: `state` (NORMAL / DEGRADED / MITIGATING / RECOVERED), `rung` (position on the ladder), `policy` (current qdisc name).
- `recent`: the last 64 delay samples, used to compare delay *before* and *after* a change.
- `bad_since / good_since / idle_since`: when each condition started being true, or None. They are used to require that a condition **persists** before acting.
- `_switch()`: records the delay over the last 3 s as `before`, moves to the new rung, calls `qdisc.apply()`, resets the timers, and prints a `>>> qdisc` event.
- `_goto()`: changes state and prints a `>>> state` event.

3.9 Controller.step(): classify this tick (controller.py lines 197-212)

```
197      def step(self, now, rate_mbit, delay_ms, inflation_ms):
198          a = self.a
199          self.recent.append((now, delay_ms))
200          saturated = rate_mbit >= a.saturation * a.rate_mbit
201          # The two thresholds are deliberately far apart (hysteresis).
202          bad = saturated and delay_ms > a.enter_ms and (inflation_ms is None or inflation_ms > a.enter_ms
           )
203          good = delay_ms < a.exit_ms and (inflation_ms is None or inflation_ms < a.exit_ms)
204          idle = rate_mbit < 0.5 * a.rate_mbit
205
206          held = lambda since: since is not None and now - since
207          self.bad_since = (self.bad_since or now) if bad else None
208          self.good_since = (self.good_since or now) if good else None
209          self.idle_since = (self.idle_since or now) if idle else None
210
211          if a.observe_only:
212              return
```

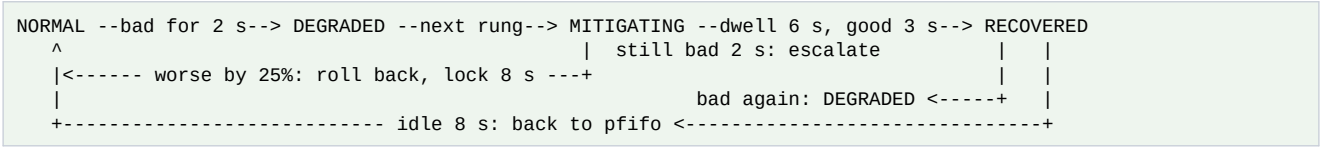
- Called every 0.5 s with the link rate, the delay (eBPF min sojourn) and the ping inflation.
- **saturated**: rate ≥ 85% of 10 Mbit. A queue can only build up when the link is full.
- **bad**: saturated **and** delay > 50 ms **and** RTT inflation > 50 ms. All three must hold, which avoids false alarms.
- **good**: delay < 20 ms and inflation < 20 ms. The gap between 50 and 20 is **hysteresis**, so values hovering near one threshold do not cause flapping.

- **idle**: rate below 50% of capacity, meaning the load is gone.
- Lines 206-209: `held(since)` gives how long a condition has been continuously true. Each `*_since` is set when the condition first becomes true and cleared when it stops.
- Line 211: with `--observe-only` (the control run) it measures and classifies but never acts.

3.10 The state machine: decide and act (controller.py lines 214-250)

```
214          if self.state == "NORMAL":
215              if now >= self.locked_until and held(self.bad_since) >= a.persist:
216                  self._goto("DEGRADED", f"standing queue for {a.persist:g}s: delay {delay_ms:.0f} ms on a
          saturated link")
217
218          elif self.state == "DEGRADED":
219              if self.rung + 1 < len(LADDER):
220                  self._switch(self.rung + 1, now, "escalate")
221                  self._goto("MITIGATING", f"holding for {a.dwell:g}s dwell")
222              else:
223                  self._goto("MITIGATING", "already on the strongest policy")
224                  self.changed_at = now
225
226          elif self.state == "MITIGATING":
227              if now - self.changed_at < a.dwell:
228                  return      # measurements still reflect the transient of our own change
229              after = self._median_delay(self.changed_at + a.dwell / 2)
230              if held(self.good_since) >= a.exit_persist:
231                  self._goto("RECOVERED", f"delay {self.before:.0f} -> {after:.0f} ms, keeping {self.polic
          y}")
232              elif self.before and after > 1.25 * self.before and self.rung != self.prev_rung:
233                  self._switch(self.prev_rung, now, f"rollback: delay got worse, {self.before:.0f} -> {aft
          er:.0f} ms")
234                  self.locked_until = now + a.cooldown
235                  self._goto("NORMAL", f"locked out for {a.cooldown:g}s")
236              elif held(self.bad_since) >= a.persist:
237                  if self.rung + 1 < len(LADDER):
238                      self._switch(self.rung + 1, now, f"{LADDER[self.rung]} did not help, escalate")
239                  elif not self.told_top:
240                      self.emit("top of the ladder and still degraded; holding")
241                      self.told_top = True
242
243          elif self.state == "RECOVERED":
244              if held(self.bad_since) >= a.persist:
245                  self._goto("DEGRADED", f"degraded again under {self.policy}")
246              elif held(self.idle_since) >= a.cooldown and not a.no_revert and self.policy != a.base:
247                  # Revert on absence of load, never on low delay: the low delay is
248                  # caused by the policy, so reverting on it would flap.
249                  self._switch(LADDER.index(a.base), now, f"link idle for {a.cooldown:g}s, back to base")
250                  self._goto("NORMAL", "load gone")
```

- **NORMAL**: if "bad" has held for 2 s (and no rollback lockout is active), go to DEGRADED.
- **DEGRADED**: move up one rung (pfifo -> fq_codel, or fq_codel -> cake) and go to MITIGATING. If already on cake, just hold.
- **MITIGATING**: first wait the 6 s **dwell**, because right after a switch the measurements show the switch itself, not the new steady state. Then compare: (a) good for 3 s, go to **RECOVERED** and keep the qdisc; (b) delay is more than 25% *worse* than before, **roll back** to the previous qdisc and lock out for 8 s; (c) still bad for 2 s, **escalate** to the next rung, or say "top of the ladder".
- **RECOVERED**: if it goes bad again, back to DEGRADED. If the link has been **idle for 8 s**, switch back to pfifo and go to NORMAL.
- Lines 247-248: it reverts when the **load** goes away, never because delay is low. Low delay is the *result* of fq_codel. Reverting on it would bring the bloat back, trigger again, and flap forever.



3.11 main(): command-line options (controller.py lines 253-278)

```
253  def main():
254      p = argparse.ArgumentParser(description=__doc__, formatter_class=argparse.RawDescriptionHelpFormatte
           r)
255      p.add_argument("--ns", default="bb-r", help="namespace of the bottleneck router ('' for none)")
256      p.add_argument("--dev", default="r-eth1", help="bottleneck egress interface")
257      p.add_argument("--parent", default="1:1", help="shaper class holding the managed leaf qdisc")
258      p.add_argument("--rate-mbit", type=float, default=10.0, help="bottleneck capacity")
259      p.add_argument("--pfifo-limit", type=int, default=1000)
260      p.add_argument("--base", choices=LADDER, default="pfifo", help="policy when nothing is wrong")
261      p.add_argument("--probe-ns", default="bb-h1")
262      p.add_argument("--probe-target", default="10.0.2.2")
263      p.add_argument("--tick", type=float, default=0.5, help="control loop period, s")
264      p.add_argument("--enter-ms", type=float, default=50, help="upper threshold: delay above this is bad"
           )
265      p.add_argument("--exit-ms", type=float, default=20, help="lower threshold: delay below this is good"
           )
266      p.add_argument("--persist", type=float, default=2, help="s a bad condition must last before acting")
267      p.add_argument("--exit-persist", type=float, default=3, help="s a good condition must last")
268      p.add_argument("--dwell", type=float, default=6, help="s to hold after a change before judging it")
269      p.add_argument("--cooldown", type=float, default=8, help="s of idle link before reverting to base")
270      p.add_argument("--saturation", type=float, default=0.85, help="fraction of capacity that counts as s
           aturated")
271      p.add_argument("--baseline-window", type=float, default=120, help="s over which min RTT is the basel
           ine")
272      p.add_argument("--observe-only", action="store_true", help="measure and detect, never change the qdi
           sc")
273      p.add_argument("--no-revert", action="store_true", help="keep the stronger policy after load ends")
274      p.add_argument("--no-ebpf", action="store_true", help="fallback: derive delay from backlog / rate")
275      p.add_argument("--no-probe", action="store_true", help="do not run the ping probe")
276      p.add_argument("--duration", type=float, default=0, help="stop after this many seconds (0 = until Ct
           rl-C)")
277      p.add_argument("--log", default="results/run.csv")
278      args = p.parse_args()
```

- Every threshold is an option, so experiments need no code changes. Defaults: enter 50 ms, exit 20 ms, persist 2 s, dwell 6 s, cooldown 8 s, saturation 85%, tick 0.5 s.
- `--observe-only` gives the control run, `--no-ebpf` uses the fallback delay estimate, and `--duration` plus `--log` are used by `run_demo.sh`.

3.12 main(): start-up (controller.py lines 280-318)

```
280     if os.geteuid() != 0:
281         sys.exit("run as root (loading eBPF and changing qdiscs both need it)")
282     os.makedirs(os.path.dirname(args.log) or ".", exist_ok=True)
283
284     qdisc = Qdisc(args.ns, args.dev, args.parent, args.pfifo_limit)
285     qdisc.stats()
286     qdisc.apply(args.base)
287
288     observer = None
289     if not args.no_ebpf:
290         try:
291             observer = Observer(args.dev)
292         except Exception as e:      # noqa: BLE001 - any load failure means fallback
293             print(f"!! eBPF observer failed to load ({e}); falling back to backlog/rate", flush=True)
294
295     t0 = time.monotonic()
296     probe = None
297     if not args.no_probe:
298         probe = PingProbe(args.probe_ns, args.probe_target, t0, args.log.replace(".csv", "") + ".ping.cs
      v",
299                           args.baseline_window)
300         probe.start()
301
302     events = []
303     def emit(msg):
304         events.append(msg)
305         print(f"  >>> {msg}", flush=True)
306
307     ctl = Controller(args, qdisc, emit)
308     stop = threading.Event()
309     signal.signal(signal.SIGINT, lambda *_: stop.set())
310     signal.signal(signal.SIGTERM, lambda *_: stop.set())
311
312     fields = ["t", "state", "policy", "rate_mbit", "backlog_bytes", "qdelay_derived_ms", "sojourn_min_ms
      ",
313               "sojourn_avg_ms", "sojourn_max_ms", "rtt_ms", "rtt_base_ms", "inflation_ms", "drops", "flo
      ws", "event"]
314     mode = "observe-only" if args.observe_only else "adaptive"
315     source = "eBPF sojourn (min per 100 ms window)" if observer else "backlog / rate (no eBPF)"
316     print(f"{mode} controller on {args.dev}: {args.rate_mbit:g} Mbit, base {args.base}, delay signal: {s
      ource}")
317     print(f"{'t':>6} {'state':<10} {'qdisc':<8} {'rate':>6} {'backlog':>8} {'derived':>8} {'sojourn':>8}
       "
318           f"{'rtt':>7} {'infl':>6} {'drops':>6}  top flow")
```

- Lines 280-281: must be root, because loading eBPF and changing qdiscs both need it.
- Lines 284-286: create `Qdisc`, check the testbed exists, and start from a known state (pfifo).
- Lines 288-293: load the eBPF `Observer`. If that fails (e.g. no BCC), fall back to the delay estimated from backlog / rate instead of crashing.
- Lines 295-300: start the ping probe thread.
- Lines 302-305: `emit()` prints the `>>>` event lines and remembers them for the CSV.
- Lines 307-318: create the `Controller`, make Ctrl-C stop cleanly, and print the table header.

3.13 main(): the 0.5 s control loop (controller.py lines 320-362)

```
320     with open(args.log, "w", newline="") as f:
321         w = csv.DictWriter(f, fieldnames=fields)
322         w.writeheader()
323         root, _ = qdisc.stats()
324         prev_bytes, prev_t, drops, prev_leaf = root["bytes"], t0, 0, (qdisc.handle, 0)
325         while not stop.wait(args.tick):
326             now = time.monotonic()
327             t, dt = now - t0, now - prev_t
328             if args.duration and t > args.duration:
329                 break
330             root, leaf = qdisc.stats()
331             rate = (root["bytes"] - prev_bytes) * 8 / dt / 1e6
332             prev_bytes, prev_t = root["bytes"], now
333             # Leaf counters restart whenever the leaf is replaced.
334             if prev_leaf[0] == qdisc.handle:
335                 drops += max(0, leaf["drops"] - prev_leaf[1])
336             else:
337                 drops += leaf["drops"]
338             prev_leaf = (qdisc.handle, leaf["drops"])
339             derived = leaf["backlog"] * 8 / (args.rate_mbit * 1e6) * 1e3
340 
341             soj = observer.sojourn() if observer else None
342             nflows, top = observer.top_flows(dt) if observer else (0, [])
343             rtt, base = probe.sample(t) if probe else (None, None)
344             inflation = rtt - base if rtt is not None and base is not None else None
345             delay = soj[0] if soj else derived
346 
347             n_events = len(events)
348             ctl.step(t, rate, delay, inflation)
349             fmt = lambda v, nd=1: "" if v is None else f"{v:.{nd}f}"
350             w.writerow({
351                 "t": f"{t:.2f}", "state": ctl.state, "policy": ctl.policy, "rate_mbit": f"{rate:.2f}",
352                 "backlog_bytes": leaf["backlog"], "qdelay_derived_ms": f"{derived:.1f}",
353                 "sojourn_min_ms": fmt(soj[0] if soj else None), "sojourn_avg_ms": fmt(soj[1] if soj else
          None),
354                 "sojourn_max_ms": fmt(soj[2] if soj else None), "rtt_ms": fmt(rtt), "rtt_base_ms": fmt(b
        ase),
355                 "inflation_ms": fmt(inflation), "drops": drops, "flows": nflows,
356                 "event": " | ".join(events[n_events:]),
357             })
358             f.flush()
359             ms = lambda v: "-" if v is None else f"{v:.0f}ms"
360             print(f"{t:6.1f} {ctl.state:<10} {ctl.policy:<8} {rate:5.1f}M {leaf['backlog'] // 1000:6d}kB
         "
361                   f"{ms(derived):>8} {ms(soj[0] if soj else None):>8} {ms(rtt):>7} {ms(inflation):>6} {d
        rops:6d}  "
362                   f"{top[0] if top else ''}", flush=True)
```

- Line 325: wake every `--tick` seconds until stopped or `--duration` is reached.
- Lines 330-332: **link rate** = increase in bytes sent by the root shaper x 8 / elapsed time.
- Lines 333-338: running total of **drops**. The leaf qdisc's counters restart from 0 whenever it is replaced, so the code checks whether the handle changed.
- Line 339: **derived delay** = backlog bytes / link rate. This is the fallback estimate, shown for comparison.
- Lines 341-345: read the eBPF sojourn, the top flows and the ping RTT, then compute inflation. **The delay used for decisions is the eBPF min sojourn**, or the derived value if eBPF is off.
- Line 348: `ctl.step(...)`: detect, decide, and act if needed.
- Lines 350-358: write one CSV row per tick (`results/*.csv`). `plot.py` uses these.
- Lines 359-362: print the live table row seen during the demo.

3.14 Exit (controller.py lines 364-372)

```
364      if probe:
365          probe.stop()
366      if ctl.policy != args.base:
367          qdisc.apply(args.base)      # leave the interface in a known state
368          print(f"  >>> exit: restored {args.base}")
369
370
371  if __name__ == "__main__":
372      main()
```

- Stop the ping process, and if a stronger qdisc is still installed, put pfifo back so the interface is left in a known state.

4. One packet and one tick, end to end

Step	Where	What happens
1	iperf3 in bb-h1	Sends TCP at full speed toward 10.0.2.2. The router forwards it to r-eth1.
2	observer.bpf.c L87	Packet handed to the qdisc: timestamp stored in enq_ts, bytes added to flows.
3	kernel qdisc	Under pfifo the packet waits behind up to 1000 others, up to ~1 s at 10 Mbit.
4	observer.bpf.c L100	Packet leaves: sojourn = now - stamp, folded into the current 100 ms slot.
5	controller.py L341	Every 0.5 s: sojourn() reads the 5 finished slots and takes the min.
6	controller.py L202	Saturated + min sojourn > 50 ms + ping inflation > 50 ms = bad.
7	controller.py L215	Bad held for 2 s: NORMAL -> DEGRADED.
8	controller.py L220, L61	Next tick: tc qdisc replace ... fq_codel. State MITIGATING.
9	controller.py L230	After 6 s dwell, delay good for 3 s: RECOVERED (verify).
10	controller.py L249	iperf3 ends, link idle 8 s: back to pfifo, NORMAL.

5. Likely examiner questions

Question	Answer
Why eBPF instead of just ping?	Ping only shows the problem indirectly, end to end. eBPF measures the queue wait of **every packet** at the exact place it happens, for any qdisc, with aggregation in the kernel so the overhead stays low.
Why tracepoints, not a tc-BPF program?	A tc-BPF program runs at one point (before the qdisc) and cannot see when a packet leaves. The net_dev_queue / net_dev_start_xmit pair gives true enqueue-to-dequeue time.
Why use the minimum sojourn?	A burst raises the max or average briefly. A min above the threshold in every window means the queue never drained: a standing queue (CoDel's idea).
Why three conditions?	To avoid false alarms. Delay without saturation is not bloat, and the ping check confirms that users actually feel the delay.
What stops it flapping?	Hysteresis (50 / 20 ms), persistence (2-3 s), a 6 s dwell after each change, an 8 s lockout after rollback, and reverting only when the load is gone.
Is it safe to run in the kernel?	Yes. The verifier checks the eBPF before it is loaded. The program only reads packets and updates maps. It never changes or drops traffic.
What if eBPF fails to load?	controller.py falls back to delay = backlog / rate (line 293).